

小芽：宠物对战游戏里的 LLM 教练

设计与实现说明

腾讯 IEG 面试题提交稿 2026 年 9 月

本报告由 AI 辅助撰写，全部数据与结论由本人核实。正文引用的每个数字都对应仓库里一份可复现的产物；结论、失败与未完成项经本人逐条核对后写入，没有实测过的部分，文中直接写明没测过。

1 题目与取舍

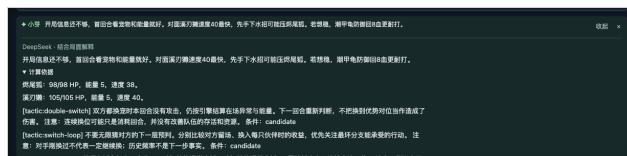
这道题要求为宠物对战游戏做一个 AI 教练，具备军师、陪练、老师三种角色感，参考方向列了 Agentic RL、RAG、ReAct、Memory Stream / Reflection，产品参考是和平精英的「小田」与王者荣耀的「灵宝」。我的理解是：交付物是教练，游戏只是产生局面的环境；三种角色感不是三种语气，而是三种不同的正确性标准——军师要判断准确且可解释，老师只能依据当时可见的信息，陪练要记得住并且不越界。

第一个动作是划边界，而不是写代码。三件事本次不做：**美术与完整游戏内容**，规模控制在 14 只宠物、7 种属性、5 个关卡，这个量已经足够产生「抢先收尾还是留能量」「换宠承伤还是原地治疗」这类真实取舍；**联网 PVP**，本地对战是同机分屏轮流操作，服务端仍然接受客户端传来的状态快照，不是权威对局服务，这条差距是架构问题而不是工程量问题；**模型权重训练**，这个项目的重心是 agent 侧的工程——工具调用、检索、记忆、验收，训练模型不在其中，本机没有可用的 CUDA 后端只是顺带的事实。

三种角色我全部做完了，题目要求的是至少闭环一项。它们的难点并不重叠：军师的难点在分工，老师的难点在时间口径，陪练的难点在克制；只做一种，后两类问题不会暴露出来。

2 交付了什么

军师。引擎枚举双方全部合法行动，对每一种组合跑一遍共享结算器，模型只负责把算好的事实讲成玩家能理解的取舍；双方同速时两种出手顺序各算一次，搜索不会偷偷利用随机数。局内主动提示也归军师而不是陪练，触发条件是代码里的五条：伙伴倒下、在两个选项间来回犹豫、真实枚举出来的明显策略错误、长停留，以及血量低于 35% 而背包里还有药；「安静档」是玩家自己设的一个档位：这一档下教练不主动开口，只在玩家提问时回答，它不是关闭教练，而是把教练改成被动。它和「本局不再提醒」永远优先于上面任何一条触发条件。



上图的展开区是这条提示的计算依据：双方当前血量与能量，以及它引用的两张知识卡。依据里的数字全部来自引擎枚举，模型只写最上面那句话。

陪练。它的重点是陪伴。它记得住：状态从真实对战历史派生——连败几局、最近赢没赢、上一局打到第几回合、还剩几瓶药、本命是哪只、上次玩是几天前；这些字段决定它用哪一档语气，档位本身会作为证据写进模型输入，所以它说出的每句话都能回溯到一条本机记录。它在场：对局中间会自己插一句看法，不提问、不给指令。它有情绪，也会吐槽局面，比如「潮甲龟连着 2 个回合被草系按着打，我看得有点急」。



气泡在战斗页左下角，独立于聊天面板：不打开聊天也能看见它，可以点掉，点掉之后本局静音。每局主动搭话有上限，最近七天的关闭次数会降低这个上限，四次之后直接归零。它不评价玩家水平、不给战术指令、不说教，这三条由 `checkCompanionRestraint` 在出句后扫描。

2.1 老师

老师不只是复盘。它盯着玩家看哪一个技能不动就讲哪一个：属性、能耗、算在当前目标身上多少伤害、打完剩几豆、什么条件下比手里别的选项更合适，但不催出招。每回合结束给这一回合的事实；整局结束给复盘，从减员和生命变化里挑三个回合逐个分析。小测按真实面板出变式题（「速度 38 加一次敏捷 +3，对速度 41 的先手还是后手，还是无法确定」），选项里带「不确定」。培养建议是它唯一涉及成长的入口，给出加点收益与这次加点是否改变先手关系。

真正花心思的是教学账本。它记三件事：讲过哪一课、哪一课被标回未掌握、这一课有几个被提示过和没被提示过的行动。讲过又没有新反证就不重复；只有军师在局内独立观察到三次同类失误（**RELEARN**：至少 3 次行动、合理率低于 50%）才标回未掌握，并能说出「这一课教过，但之后 4 次独立行动里有 3 次不合理」。重新讲过就消费掉，再教要靠新出现的独立失误。

这里有一个容易踩的坑，我是从文献里看到的。讲 assistance dilemma 的那篇论文报告，他们早期的预测器没把「学生用过提示」算进去，结果因为提示本身会把学生推向高效步骤，用过的学生反而被判成「不需要帮助」；他们的修正就是把两者分开。小芽里是同一件事：判断玩家是否掌握某个知识点时，只用他没有被提示过的时候做出的选择。

复盘一律使用回合开始时的公开状态，不使用对手实际打出的那一招——否则任何一次失败都能被解释成「你当时选错了」。改掉对手的实际行动，复盘结论不变，这条有测试。

这套账本是自建的计数与规则，不是心理测量模型：只判断教过没有、这次要不要再教，不估计掌握概率。

`npm start` (Node.js 20+, 无第三方依赖)。浏览器打开 `http://127.0.0.1:8765/`。顶部两个入口：训练按关卡挑战固定对手，对局里对手自由配队，也可以是同机分屏的两名真人，两边各有一条教练。**不接模型也能完整游玩，规则建议、复盘、培养建议和小测都不依赖模型；接入模型之后，这些解释改由模型生成。**密钥只留在后端进程内存里，不落盘。

语音已整体停用，联网 PVP 未做，依赖真人被试的验收没有完成，浏览器慢响应只有替换请求级别的验证。逐条写在第五节。

3 协作中的难点

实现由子 agent 写，我做三件事：定目标、定验收标准、判断结论对不对。下面四类问题是我在验收里抓出来的。

3.1 测试通过而功能没有生效

全项目出现最多的一类问题是：代码看着对、测试全绿、功能根本没有生效。四个实例：

- 子 agent 改服务端时漏了静态资源白名单里的新模块 `rules.js`，`/rules.js` 返回 404，`app.js` 的 `import` 图整体失败，页面元素没有渲染出来。
- 子 agent 给陪练写了 `notify()`，全仓库没有调用点。主动气泡写完了，从不出现。
- 子 agent 改军师时留下一个未定义变量，每秒抛一次 `ReferenceError`，浏览器日志里攒了 75 条。
- 敌方补位「决定了却无人提交」：子 agent 选好了合法换宠、存进变量就返回，而玩家那条路径又拒绝替它提交，界面永久冻结在「正在出招…」。

这四次全部是子 agent 交付的缺陷，没有一次是读代码读出来的，全部是实际跑起

来才看见的。

我把验收换成了两层检查。第一层是 `wiring.test.js`: `app.js` 里每个 `$('#id')` 都必须在 `index.html` 或 `app.js` 自建的元素里存在，死函数清单必须恰好是两个已知遗留，面试题要求的每项能力都必须在 `app.js` 里找得到入口；它第一次运行就找出一处指向已删除元素的残留引用。第二层是冒烟测试，用无头浏览器真打一局训练赛，断言营地渲染出伙伴卡、对局能打到结果、控制台零报错；它不进 `npm test`，因为需要浏览器的检查不该当成必过门。

这一层有假阳性。有一次它报「最长连续无可点 14 次」，看上去快卡死了，查下去是它从未设置播放速度，每一回合都在放动画、按钮本来就处于禁用状态。

3.2 检查本身也要被检查

玩家文案的术语检查写出来之后，我做了负向验证：把禁止词表里的词植入一句玩家可达的文案，测试必须失败；移除，测试必须通过。这一步让一个原本看不出来的缺口暴露了——检查取局内提示时没有传事件参数，于是「明显策略错误」那条理由对应的正文根本不在覆盖范围内。

第一次修补的方向也是错的：手工造一个事件参数传进去，负向验证立刻戳穿，把违规词放回源码，检查照样通过——那段正文要等枚举真的算出一次事故才会生成。最后换成静态扫描，逐行取 `coach/` 下的中文字面量对词表，不依赖任何运行时路径；它随即又扫出四处运行时检查没覆盖到的真泄漏。

3.3 代码是对的，错的是统计

有一处缺陷代码完全正常，错的是统计口径：玩家补位的那一回合本来就不该出招，却被计成一次「引擎兜底」，方向恰好让对手 `agent` 显得比实际更差。同类的一处是犹豫判定用了墙钟时间，玩家中途去操作对方面板的 13 秒被算成「犹豫了 13 秒」。这类错误不会让任何测试失败。

3.4 哪些判断我不外包

有一份评测结果文件存在，我打开只有两行，是更早一次局部验证的残留；此后我逐条看实际数据，不看文件名。

另一类更容易漏：子 `agent` 写下的「某方案没用」往往只对了一半。检索实验里最好的一臂确实没有显著赢过基线，但拆开四条臂能看到排序指标确实变好了。

4 实现中的难点

这一节的主线是把「算」和「说」分开：引擎负责对，模型负责讲。下面几个取舍都是这条主线的推论，也都是我改错过的地方。

4.1 数字不能让模型算

第一版我让模型看着局面直接给建议，得到的回答里数字是错的，而且稳定地错：编出不存在的技能名，算错伤害，在两个都能击倒的技能里挑面板数字大的那个。改法是把职责切开，引擎枚举、模型解释。

同一个残血目标，火花直接伤害 52，追猎 75，两者都够击倒，都耗 2 点能量。只看面板，很自然会建议选追猎。但这是错的：追猎的额外伤害来自「目标已灼烧」这个前提，对方吃药、换宠、防御都会让前提失效。这种判断交给模型是猜，交给枚举是算。

切开之后还剩两类数字校验抓不到的错。第一类是道具名：游戏里只有回复药、净化药、能量果三样，模型会把净化药说成「解药」、能量果说成「以太」——数字全对，引用全对，只有名字是错的，因为没有数字可校验。第二类是因果：伙伴倒下时，它这一回合已经排好的行动会被取消，模型会把这次取消描述成「命中了」。两类都靠新增检查解决：一份别名黑名单，命中即判不合格；读回合事件，某个行动被记为取消时，正文里就不能出现它造成了伤害。

最后一类更隐蔽：模型读到的规则本身就是旧的。原来的属性相克表不对称，我换成两个独立的三环；军师、经验层、规则页都从同一张表读，自动跟着走了，但把规则抄成自然语言的副本不会。我改掉了手写的那张边界卡，扫第二遍才发现六张由脚本生成的属性卡里仍然写着旧数值，而它们正是检索喂给模型的地面事实：教练会照着一张引擎已经不再实现的规则回答，测试全绿，因为没有任何测试盯这句话。根因是那句话在生成脚本里是硬编码的字符串，这一轮改成从规则对象取数。这类副本我手工找漏了三次。

4.2 「该不该调工具」不该由模型决定

工具决策我做过三轮真实评测，同一套 44 条预注册用例，全部真实 API 调用，变差的那一轮也保留。三轮依次改的是：原始提示词；把「默认停止」写进提示词并点名哪几类必须调；给模型加一个「必须调用」的字段。第三轮已回退。

指标	第 1 轮	第 2 轮	第 3 轮
多余工具调用率	75.0%	33.3%	13.6%
漏调率	0.0%	21.9%	77.3%
严格正确率	31.25%	50.0%	31.8%
模型自停率	3.23%	96.8%	—
延迟 p50	4284 ms	2798 ms	2154 ms

第 2 轮不是纯改善：多余调用从 75.0% 降到 33.3%，漏调却从 0 升到 21.9%。第 3 轮更差：我加那个字段本意是补上三类硬需求，实际等于告诉模型「这个字段是空就不必调」，而它在大多数用例上都是空的。

三轮的摆动说明问题不在措辞，在分工。数据给出的分界很清楚：模型**选哪个工具**很准（第 2 轮在「预期应调且确实调了」的 12 条上是 12/12），但**判断该不该调**只有一半正确。所以第 4 轮不是继续改提示词，而是把政策写进代码：证据包里已经带着当前局面和最近回合事件，问当前状态的不用再查，只有结构上不含的四类才需要查；判定需要时由代码直接执行第一步，再问模型要不要继续，判定不需要时整个工具循环跳过。

44 条里只有 32 条可判定，每格 1 条、单次运行，上表所有差值都没有置信区间；即使第 2 轮，严格正确率也只有 50%。而且这 50% 里有一部分不是模型错——预注册标准认为「我这只还剩多少血」应该调一次读状态，但证据包里本来就带着当前局面，第 2 轮那 7 条漏调里有 6 条属于这一类，是两种政策的差异。产品必须选一个并说清楚，我选了「包里已有的事实不重复取」。

4.3 检索没有赢过关键词

我的假设是：在词项检索之上再加一层领域概念扩展，口语化提问的命中率会提高。为了验证它，我建了一个四臂对照：不检索；词项检索（中文二元词项加权 + IDF）；词项加领域概念扩展；以及一个把反例文本提为一等字段的第四臂。128 条查询（112 条有正解、16 条越界），离线跑，不调用模型。

四臂都是本机可查的词项方法，**没有一个用到向量**——领域概念扩展是一份可审阅的领域词表，不是预训练 embedding。仓库里另有一个真正的多语言向量检索实现（Python 侧 ‘scripts/semantic-worker.py’，用 ‘model.encode’ 编码知识卡），它不在这组 128 条实验里，两者的数据集与实现都不同，不能混在一张表里比较。

臂	Hit@3	MRR	负例拒答
不检索	0.00%	0.0000	16/16（平凡结果）
纯关键词（现默认）	88.39%	0.8007	9/16
混合（词项 + 语义）	90.18%	0.7996	9/16
反例字段加权	91.07%	0.8244	9/16

结果是没有任何一个臂显著赢过关键词基线。最好的一臂只高 2.68 个百分点（112 条正解里 5 胜 2 负），McNemar 精确检验 $p = 0.4531$ ，配对 bootstrap 95% 置信区间 $[-1.79, +7.14]$ 个百分点，跨零。

我差点在这儿犯一个错。看到混合没赢，我的第一反应是「检索没用，RAG 是伪需求」。把四臂拆开才发现不是：第四臂的 MRR 从 0.8007 升到 0.8244，排序确实变好了，只是这些改善都发生在 112 条正解上，一条查询的变动就能让 Hit@3 动将近一个百分点。最后我记成「这次测量没测出差别」，既没有换成向量检索，也没有宣称 RAG

无效。

真正被这轮测出来的是负例拒答：三个检索臂都只有 9/16，越界提问仍然召回了卡片，原因是词项检索只要命中任意二元组就判为正分，没有最低分阈值，也没有拒答机制。这是尚未解决的问题，调权重解决不了。另外，这 128 条查询和标注都由我完成，不是独立盲标集：沿用的 20 条旧查询 Hit@3 是 81.25%，新写的 92 条是 93.48%，说明出题时我已知道卡片里有哪些词，所有绝对数字都偏乐观。

4.4 玩家不打开聊天，就等于没有教练

第一版做出来我自己试了两局，发现一件事：小芽的全部能力都在聊天面板里，而我全程没有打开过那个面板。改法是把主入口从聊天框挪到游戏事件上：开局和关键风险变化时，战斗页底部出现一行短提示，解释收在二级入口里按需展开；玩家出招之后旧提示立刻隐藏，等新局面重新评估。

副作用是提示一多就变成噪音，于是门控反而比内容更难做：玩家是否在前台、是否已经出招、是否已经忽略过、这个回合是否已经提示过、上次提示距今多久、当前风险是否构成可行动的风险，逐项判断；每局总量、同类提示只出一次、冷却时间各有上限，老师和军师还共用同一份每局配额。

讲 assistance dilemma 的那篇论文里，作者报告他们早期的预测器没有把「学生用过提示」算进去，结果因为提示本身会把学生推向高效步骤，用过的学生反而被判定成「不需要帮助」，他们的修正是把两者分开。小芽里是同一件事：判断玩家是否掌握某个知识点时，只使用他没有被提示过的时候做出的选择。我先按直觉这么设计了，后来才发现这个坑在文献里有记录。

4.5 对手交给模型，引擎只列合法行动

题目场景是 PVP，我最初的做法是让引擎的启发式当对手：轻松档随机，标准档优先伤害和治疗，挑战档枚举双方合法行动并比较一回合结果。这一轮改了——**对手的决策交给模型，引擎只负责列出有哪些合法行动。**

改的理由不是「这样显得对手更聪明」。恰恰相反，我查到的证据方向是反的：在这个游戏类型里，决定胜负的是搜索和训练，不是语言能力。变的是它回答的问题——那个结论说的是「谁更强」，而这轮要解决的是「对面得是个人」。一个按伤害公式调出来的启发式，出招永远挑不出毛病，也永远不像人。

难度因此改成只控制「给对手多少帮助」，不控制谁在决策：简单档只给合法行动列表，标准档多给一句对手自己教练的建议，挑战档再给出这一回合的枚举分数。对手那个「自己的教练」不是另写一套，而是拿同一份军师推理函数把双方对调后跑出来的。

兜底只在四条路径上出现：模型未配置、模型选了列表外的行动、超时（上限 4 秒）、网络错误；四条各有测试，且都必须在没有后端、没有密钥的前提下工作。本次干净跑

里四条都没有被真实流量碰到（兜底 0 次），因此并发压力下这条链路的上限如何，我没有测过。

胜率这一项我没有测出结论。正常节奏下玩家每回合等待 0 毫秒——对手思考的那一秒被玩家自己的思考时间吸收了；但把 24 局放在一起看，四个格子两边的胜率是 100/100、100/100、66.7/50、50/50，**对手一局都没赢过**。准确的说法是：n=6 而且对手全败，这套对照既不能支持「对手变强了」，也不能支持「打平了」，只能说没有变弱的迹象。

同一件事上我自己改错过两次。题目对军师的描述是「在对战 / 阵容场景给出决策建议」，场景设定写的也是 PVP，军师本来就该在对战里工作；我第一版定下的判据却是「只要是对局模式，教练一律不介入」，理由是两人共用一块屏、给一方提示不公平，我甚至把它写成「公平性设计」当亮点——这等于把主功能在题目指定的场景里禁用了。第二次改成「对手是真人时闭麦、对手是电脑时照常」，又被指出不自洽：电脑对手存在的意义就是模拟真人，行为就该一致，否则这个模拟本身就是假的。最终改成本地对战一律允许教练，只保留线上竞技的闭麦开关；真正的底线不是「整局不说话」，而是「不读取对方待执行的动作」，那条一直没破，也有测试。

5 结果

项目	值
宠物 / 属性 / 关卡	14 只 / 7 种（普通系双向中性） / 5 个
知识卡	93 张（49 张战术卡 + 44 条引擎参考卡）
自动测试	npm test: 257 项全部通过，18 个测试文件
真实模型评测	44 条预注册用例 × 3 轮 = 132 次真实调用
检索对照	128 条查询 × 4 臂，离线，不调用模型
平衡矩阵	5,796 场 / 398 个臂，规则版本 0.6，本机 185 秒
对手 agent 实测	4 格 × 6 局，543 次真实决策

三种角色全部闭环，且都不依赖模型可用——模型不可用时退回规则结论，功能不消失。

用新的相克表重跑 5,796 场，结论是没有哪只宠物被证明明显更强：同属性的六组配对在主要对照里全部未分离；旧规则下唯一统计显著的同类失衡（雷系两只相差 33.3 个百分点、置信区间不重叠）缩到 8.3 个百分点，区间重叠，消失了；九套阵容的胜率极差从 63.0 个百分点收窄到 35.2 个百分点。但矩阵能证明的只是「那条差距消失了、离散度收窄了」，**不能**证明「属性平衡整体变好」——那需要跨属性对照和消融实验；每格 12 到 60 场，小于约 12 到 15 个百分点的胜率差分不开，玩家侧也不是真人。

检索没有证据表明任何一臂更好，只能说没有证据表明更差。对手 agent 的延迟可用（p50 约 1.05 秒，正常节奏下玩家无感），但 24 局胜负分不开两臂，关于「变强」没有结论。

仓库里另有两个本机可跑完的小实验：一个用表格 Q-learning 学「教练何时开口」，一个冻结骨干、只更新 1152 个参数学「该调哪个工具」。它们是这两个窄任务的证据，不能读成「训练了模型」——前者赢在打扰成本更低而不是教得更好，后者的 16 条测试全是英文二选一。

5.1 没有做的

项目	状态与原因
浏览器语音	已停用。本想照灵宝那样让教练出个声，接上之后语音模型一直没调好（断句、音色都不对），于是放弃并整体停用；停用之后这一块也没有可观察对象
联网 PVP	未做。需要服务端权威对局，属于架构问题，不是工程量问题
模型权重训练	未做。本项目的重心是 agent 侧的工程，训练不在其中；本机也没有可用的 CUDA 后端
真人被试相关验收	未做。无提示迁移、学习效果观察、分屏双人验收都需要真人被试
浏览器慢响应验收	未做。本机延迟只有 2-4 秒，构造不出可控的慢网
独立标注的质量评测	未做。128 条检索查询自写自标，绝对数字偏乐观
演示录像	未做。本机没有可见桌面与音频，只生成了截图序列加文字说明

接下来优先做三件：给检索加最低分阈值与拒答机制并重测负例拒答率；在看结果之前重新标注评测集；给对手 agent 换一个更强的阵容，让胜负这一项真的能分出差别。

6 索引与复现

仓库已发布，公开可克隆：<https://github.com/SeRendizc/pet-game-coach> (256 个文件，无第三方运行时依赖)。克隆之后 `npm start` 可完整游玩，`npm test` 可复现正文里除模型实验之外的全部结论。

- `npm start`: 启动，默认 127.0.0.1:8765
- `npm test`: 257 项自动测试
- `npm run test:smoke`: 无头浏览器真打一局
- `npm run build:knowledge`: 从引擎重新生成知识卡
- `npm run eval:retrieval`: 检索四臂对照

正文数字的来源：规模与测试数取自 `npm test` 的现场输出；检索对照、工具决策三轮、平衡矩阵与对手 agent 实测分别见 `reports/` 下的 `retrieval-extended.json`、

`live-model-eval*.json`、`balance-matrix.json` 与 `opponent-agent.json`。宠物数、属性数与知识卡数都能用一行命令现场复算，README 里写明了命令；模型实验需要自己的密钥。